

EXPERIMENT NO: 3

Name of the student: Shraddha Gajanan Desai **Roll No:** 18

Division: A **Batch:** A

Aim of the Experiment: Implementation of RSA Cryptography Algorithm.

Learning Objectives:

- To understand the concept of public key cryptosystem.
- To understand the RSA cryptosystem.
- To understand RSA Digital signature scheme.
- To understand the possible attacks on RSA cryptosystem.

Lab Outcomes: Students will be able to apply the cryptosystem to ensure confidentiality and integrity of information.

Tools & Software Required:

- Operating System: Ubuntu Linux / Windows
- Python

Theory:

RSA is a public-key cryptography algorithm invented by:

- Ron Rivest
- Adi Shamir
- Leonard Adleman

It is based on the mathematical difficulty of factoring large prime numbers.

RSA uses:

- Public Key (e, n) → used for encryption
- Private Key (d, n) → used for decryption

Generation of RSA Key Pair

Each person or a party who desires to participate in communication using encryption needs to generate a pair of keys, namely public key and private key. The process followed in the generation of keys is described below –

- Generate the RSA modulus (n)
 - Select two large primes, p and q.
 - Calculate $n=p*q$. For strong unbreakable encryption, let n be a large number, typically a minimum of 512 bits.
- Find Derived Number (e)
 - Number e must be greater than 1 and less than $(p - 1)(q - 1)$.
 - There must be no common factor for e and $(p - 1)(q - 1)$ except for 1. In other words two numbers e and $(p - 1)(q - 1)$ are coprime.
- Form the public key

The pair of numbers (n, e) form the RSA public key and is made public.

- Interestingly, though n is part of the public key, difficulty in factorizing a large prime number ensures that attacker cannot find in finite time the two primes (p & q) used to obtain n. This is strength of RSA.
- Generate the private key
 - Private Key d is calculated from p, q, and e. For given n and e, there is unique number d.
 - Number d is the inverse of e modulo $(p - 1)(q - 1)$. This means that d is the number less than $(p - 1)(q - 1)$ such that when multiplied by e, it is equal to 1 modulo $(p - 1)(q - 1)$.
- This relationship is written mathematically as follows – $ed = 1 \bmod (p - 1)(q - 1)$
- The Extended Euclidean Algorithm takes p, q, and e as input and gives d as output.

Example

An example of generating RSA Key pair is given below. (For ease of understanding, the primes p & q taken here are small values. Practically, these values are very high).

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

- Let two primes be $p = 7$ and $q = 13$. Thus, modulus $n = pq = 7 \times 13 = 91$.
- Select $e = 5$, which is a valid choice since there is no number that is common factor of 5 and $(p - 1)(q - 1) = 6 \times 12 = 72$, except for 1.
- The pair of numbers $(n, e) = (91, 5)$ forms the public key and can be made available to anyone whom we wish to be able to send us encrypted messages.
- Input $p = 7$, $q = 13$, and $e = 5$ to the Extended Euclidean Algorithm. The output will be $d = 29$.
- Check that the d calculated is correct by computing $de = 29 \times 5 = 145 = 1 \text{ mod } 72$
- Hence, public key is $(91, 5)$ and private key is $(91, 29)$.

Encryption and Decryption

Once the key pair has been generated, the process of encryption and decryption are relatively straightforward and computationally easy.

RSA Encryption

- Suppose the sender wish to send some text message to someone whose public key is (n, e) .
- The sender then represents the plaintext as a series of numbers less than n .
- To encrypt the first plaintext P , which is a number modulo n . The encryption process is simple mathematical step as –

$$C = P^e \text{ mod } n$$

- In other words, the ciphertext C is equal to the plaintext P multiplied by itself e times and then reduced modulo n . This means that C is also a number less than n .
- Returning to our Key Generation example with plaintext $P = 10$, we get ciphertext $C = 10^5 \text{ mod } 91 = 105 \text{ mod } 91$

RSA Decryption

- The decryption process for RSA is also very straightforward. Suppose that the receiver of public-key pair (n, e) has received a ciphertext C .
- Receiver raises C to the power of his private key d . The result modulo n will be the plaintext P .

$$\text{Plaintext} = C^d \text{ mod } n$$

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

- Returning again to our numerical example, the ciphertext $C = 82$ would get decrypted to number 10 using private key 29 –

$$\text{Plaintext} = 82^{29} \bmod 91 = 10$$

Program:

RSA Algorithm Implementation (User Input Version)

```
print("A18_Shraddha")
```

Function to calculate GCD

```
def gcd(a, b):
```

```
    while b != 0:
```

```
        a, b = b, a % b
```

```
    return a
```

Extended Euclidean Algorithm for Modular Inverse

```
def mod_inverse(e, phi):
```

```
    t1, t2 = 0, 1
```

```
    r1, r2 = phi, e
```

```
    while r2 > 0:
```

```
        q = r1 // r2
```

```
        r = r1 - q * r2
```

```
        r1, r2 = r2, r
```

```
        t = t1 - q * t2
```

```
        t1, t2 = t2, t
```

```
    if r1 == 1:
```

```
        return t1 % phi
```

else:

return None

===== USER INPUT =====

p = int(input("Enter prime number p: "))

q = int(input("Enter prime number q: "))

Step 1: Compute n

n = p * q

print("Value of n (p*q):", n)

Step 2: Compute Euler's Totient Function

phi = (p - 1) * (q - 1)

print("Value of phi(n):", phi)

Step 3: Choose Public Key e

e = int(input("Enter public key e ($1 < e < \phi(n)$): "))

Check $\gcd(e, \phi) = 1$

if $\gcd(e, \phi) \neq 1$:

print("Error: e and phi(n) are not coprime. Choose another e.")

exit()

print("Public Key (e, n):", (e, n))

Step 4: Find Private Key d

d = mod_inverse(e, phi)

if d is None:

```
print("Modular inverse not found!")
```

```
exit()
```

```
print("Private Key (d, n):", (d, n))
```

```
# ===== ENCRYPTION =====
```

```
M = int(input("Enter message (integer less than n): "))
```

```
C = pow(M, e, n)
```

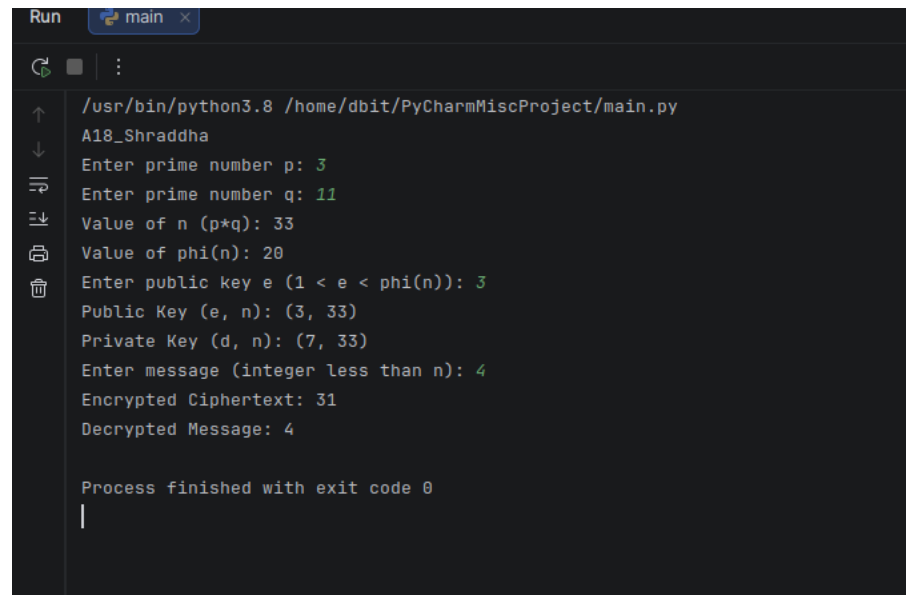
```
print("Encrypted Ciphertext:", C)
```

```
# ===== DECRYPTION =====
```

```
m = pow(C, d, n)
```

```
print("Decrypted Message:", m)
```

Output:



```
Run main x
/usr/bin/python3.8 /home/dbit/PyCharmMiscProject/main.py
A18_Shraddha
Enter prime number p: 3
Enter prime number q: 11
Value of n (p*q): 33
Value of phi(n): 20
Enter public key e (1 < e < phi(n)): 3
Public Key (e, n): (3, 33)
Private Key (d, n): (7, 33)
Enter message (integer less than n): 4
Encrypted Ciphertext: 31
Decrypted Message: 4

Process finished with exit code 0
|
```

A screenshot of your output must contain your roll number and name, otherwise it won't be considered.)

Conclusion : In this experiment, we successfully implemented the RSA Cryptography Algorithm using Python. We selected two prime numbers (p and q) from user input and calculated the modulus $n=p \times q$ and Euler's Totient Function $\phi(n)=(p-1)(q-1)$.